UNIVERSIDAD DE CALDAS	
EVALUACIÓN PARCIAL Y SOLUCIONARIO COMPLETO — PROGRAMACIÓN CREATIVA II	
Docente: Andrés Felipe Osorio Quintero	**Unidad 1:** Repaso de PC I y Conceptos Básicos de Objetos
Instrucciones: Lea cada pregunta. Sección I: Selección múltiple (1-20). Secciones II y III: Respuestas justificadas (21-30).	**Estado:** Resuelto con preguntas completas

SECCIÓN I. SELECCIÓN MÚLTIPLE CON ÚNICA RESPUESTA (PREGUNTAS 1 A 6)

1. En un sketch de p5.js se debe fijar el tamaño del lienzo una sola vez y actualizar la posición de una figura en cada fotograma. ¿Cuál distribución de instrucciones es la adecuada?

A. createCanvas() en draw() y x += vel en setup()
B. Ambas instrucciones dentro de setup()
C. createCanvas() en setup() y x += vel en draw()
D. Ambas instrucciones dentro de draw()

Respuesta correcta: C. `setup()` se ejecuta solo una vez al iniciar el programa (ideal para inicializar el lienzo con `createCanvas()`), mientras que `draw()` se ejecuta continuamente en bucle a ~60 fps, siendo el lugar indicado para actualizar coordenadas de movimiento (`x += velocidad`).

2. Una persona escribe circle(100, 20, 40) esperando ver el círculo cerca de la parte inferior del lienzo, como ocurriría en un plano cartesiano. ¿Qué observará y por qué?

A. Lo verá cerca del borde superior, origen arriba-izq y y crece hacia abajo
B. Lo verá cerca del borde inferior, eje y crece hacia arriba
C. Lo verá en el centro del lienzo por el tercer argumento
D. No lo verá, valor menor que height queda fuera

Respuesta correcta: A. En el sistema de coordenadas de pantallas y canvas de p5.js, el origen (0,0) está localizado en la esquina superior izquierda. El eje Y se incrementa positivamente hacia la parte inferior, por lo que Y=20 ubica la figura muy cerca de la parte superior.

3. En rebote.js, la condición if (x > width - 30 || x < 30) ejecuta velocidad *= -1;. Si esa línea se sustituye por velocidad = 3;, ¿qué ocurre con el círculo, que parte de x = 60?

A. Rebota igual en ambos bordes
B. Se detiene en el borde derecho
C. Rebota en el borde derecho con menor rapidez
D. Sale del lienzo por la derecha y no regresa

Respuesta correcta: D. Al cambiar la inversión del vector (`velocidad *= -1`) por la asignación fija de una magnitud positiva (`velocidad = 3`), al tocar la pared derecha la velocidad sigue apuntando a la derecha (+3), haciendo que el círculo salga indefectiblemente de la pantalla.

4. Un equipo amplía circulos_arreglos.js para que cada círculo tenga además color, radio y tiempo de vida, y luego debe eliminar los círculos que ya "murieron". ¿Qué dificultad principal enfrentará con arreglos paralelos?

A. Los arreglos no admiten guardar valores de random()
B. Debe mantener sincronizados por índice N arreglos y borrar la misma posición en todos
C. El bucle de draw() deja de funcionar con más de 4
D. Los arreglos de color solo pueden crearse fuera de setup()

Respuesta correcta: B. La arquitectura de arreglos paralelos requiere arreglos separados para `posX`, `posY`, `color`, `radio` y `vida`. Si un elemento muere en el índice `i`, se debe ejecutar `splice(i, 1)` en absolutamente todos los arreglos para evitar la desincronización de atributos.

5. Un museo quiere una instalación donde las figuras en pantalla reaccionen al movimiento de los visitantes captado por una cámara. ¿Con cuál de las aplicaciones de la programación creativa se relaciona más directamente?

A. Arte generativo con reglas y azar
B. Visualización de datos de rendimiento
C. Interfaces e interacción mediante mouse, cámara o sensores
D. Identidad visual dinámica para marcas

Respuesta correcta: C. Las aplicaciones interactivas donde la obra gráfica reacciona a insumos del mundo real (cámara, tracking de movimiento, audio) corresponden al pilar de Interfaces e Interacción.

6. Un sketch debe manejar cientos de entidades similares, cada una con su posición, su velocidad y su comportamiento propios. ¿Por qué el enfoque orientado a objetos resulta más adecuado que el procedural en este caso?

A. Separa datos de acciones y evita escribir funciones

B. Reúne datos y comportamiento en una entidad y permite crear instancias de una clase

C. Elimina la necesidad de usar arreglos en el sketch

D. Cada objeto se ejecuta en su propio ciclo draw() independiente

Respuesta correcta: B. La Programación Orientada a Objetos permite agrupar atributos (estado) y métodos (comportamiento) dentro de un molde o clase, lo que posibilita instanciar cientos de elementos encapsulados de forma limpia.

SECCIÓN I. SELECCIÓN MÚLTIPLE (PREGUNTAS 7 A 15)

7. Un sketch muestra solo dos figuras: una estática y otra que rebota. Un compañero propone crear tres clases para organizarlo. Según la unidad, ¿cuál es la valoración más acertada?

A. Indispensable, todo sketch p5.js requiere clases

B. Recomendable, las clases reemplazan lo procedural

C. Inútil, JS no combina clases con funciones sueltas

D. Innecesario, aportan valor con varias entidades similares con estado propio

Respuesta correcta: D. Diseñar tres clases para un programa trivial con solo dos figuras introduce abstracciones e hiper-estructuración innecesaria. La POO aporta verdadero valor cuando hay multiplicidad de entidades similares o complejidad de estado.

8. En un sketch existen las clases Circulo, Cuadrado y Estrella. Al recorrer el arreglo se invoca mostrar() sobre cada objeto y cada uno se dibuja a su manera. ¿Qué pilar de la POO ilustra esta situación?

A. Polimorfismo, cada clase responde a su manera al mismo mensaje

B. Herencia, reutilizan la estructura base

C. Encapsulamiento, controla el acceso a datos

D. Abstracción, representa características relevantes

Respuesta correcta: A. El polimorfismo permite enviar la misma orden/mensaje (`mostrar()`) a objetos de distintas clases y que cada uno reaccione implementando su propia versión concreta de pintado.

9. Se describe: "un pez nada hacia la derecha, tiene una velocidad y un color, y se detiene al tocar la pared". Aplicando la heurística de sustantivos, características y verbos, ¿qué diseño es el correcto?

A. Clase nadar; atributos Pez y pared; métodos velocidad/color

B. Clase Pez; atributos nadar()/detenerse(); métodos velocidad/color

C. Clase Pez; atributos velocidad y color; métodos nadar() y detenerse()

D. Clase velocidad; atributos Pez y color; métodos nadar()/pared

Respuesta correcta: C. Los sustantivos clave identifican la Clase (`Pez`), las características/adjetivos son los Atributos (`velocidad`, `color`) y las acciones/verbos son los Métodos (`nadar()`, `detenerse()`).

10. Una clase Juego mueve las figuras, las dibuja, guarda los puntajes en un archivo y calcula estadísticas de las partidas. ¿Qué recomienda la unidad frente a este diseño?

A. Mantenerla así, agrupar todo simplifica el programa

B. Dividirla en clases más pequeñas de una sola responsabilidad

C. Convertirla en objeto literal

D. Pasar métodos a funciones sueltas

Respuesta correcta: B. Principio de Responsabilidad Única (SRP). Una clase "Dios" que hace todo es difícil de mantener y probar. Debe separarse en clases especializadas (e.g., `Renderizador`, `GestorPuntaje`, `Fisica`).

11. Al evaluar typeof Burbuja sobre una clase declarada con class, el resultado es "function". ¿Cómo se explica este comportamiento?

A. Es una función común que se llama sin new

B. JS trata toda clase como objeto literal

C. Se incorporaron como función de p5.js

D. class es una capa de sintaxis (azúcar sintáctico) sobre prototipos

Respuesta correcta: D. Las clases en JavaScript ES6 no son una sintaxis OOP nativa de bajo nivel, sino una capa más amigable (azúcar sintáctico) construida sobre el sistema clásico de funciones constructoras y prototipos.

12. Dado el siguiente código, donde el constructor de Burbuja asigna this.velY = -1.5 y el método flotar() ejecuta this.y += this.velY, ¿qué valores de y tendrán b1 y b2 al final?

```
const b1 = new Burbuja(120, 300, 24, "turquesa");
const b2 = new Burbuja(260, 340, 40, "ámbar");
b1.flotar();
```

A. b1.y vale 298.5 y b2.y vale 340

B. b1.y vale 298.5 y b2.y vale 338.5

C. b1.y vale 300 y b2.y vale 340

D. b1.y vale 340 y b2.y vale 298.5

Respuesta correcta: A. El método `flotar()` solo fue invocado en la instancia b1 (300 + (-1.5) = 298.5). El objeto b2 nunca ejecutó su método, por lo que su estado permanece congelado en 340.

13. Un estudiante escribe la siguiente clase y luego invoca flotar() sobre una instancia. ¿Qué sucede y por qué?

```
class Burbuja { constructor(x, y) { this.x = x; this.y = y; } flotar() { y += 2; } }
```

A. Incrementa el atributo y de la instancia

B. Falla, y se busca como variable local/global y no como atributo de objeto

C. Crea un atributo nuevo y dentro de la instancia

D. Incrementa y en todas las instancias

Respuesta correcta: B. En JavaScript es estricto e indispensable anteceder el prefijo `this.` para hacer referencia a las propiedades del objeto. Al escribir `y += 2` sin `this`, el intérprete busca una variable y local o global no definida, arrojando un ReferenceError.

14. Para la clase Burbuja se plantean tres métodos: area() devuelve Math.PI * this.radio ** 2, reiniciar() vuelve a ubicar this.y en la parte inferior y mostrar() dibuja el círculo. ¿Cuál clasificación es correcta?

A. area() modifica estado, reiniciar() consulta, mostrar() calcula

B. area() produce efecto externo, reiniciar() calcula, mostrar() modifica

C. area() consulta, reiniciar() produce efecto externo, mostrar() modifica

D. area() consulta/calcula, reiniciar() modifica estado y mostrar() produce efecto externo

Respuesta correcta: D. `area()` calcula y retorna un valor sin cambiar nada; `reiniciar()` modifica directamente el estado interno (`this.y`); `mostrar()` dibuja en la pantalla (produce un efecto secundario en la UI).

15. Una persona pasa c.incrementar como función de respuesta a un evento y obtiene TypeError: Cannot read properties of undefined (reading 'valor'). ¿Cuál es la causa y una corrección adecuada?

A. Perdió referencia a this; corregir con c.incrementar.bind(c) o función flecha

B. Falta declarar valor con let dentro del método

C. El método debió ser estático con static

D. La clase se llamó sin el operador new

Respuesta correcta: A. Al pasar un método como callback, este se descontextualiza de la instancia perdiendo su `this`. Se soluciona vinculando explícitamente el contexto mediante `.bind(c)` o envolviéndolo en una función flecha.

SECCIÓN I. SELECCIÓN MÚLTIPLE (PREGUNTAS 16 A 20)

16. ¿Qué imprime el siguiente código?

```
const a = new Burbuja(100, 200, 20, "turquesa");
const b = a;
b.radio = 50;
const c = new Burbuja(100, 200, 20, "turquesa");
console.log(a.radio, a === c);
```

A. 20 false
B. 20 true
C. 50 false
D. 50 true

Respuesta correcta: C. Los objetos se asignan por referencia: b apunta a la misma dirección que a, por lo que `b.radio = 50` cambia `a.radio` a 50. Luego, `a === c` evalúa `false` porque son dos objetos instanciados en posiciones distintas de memoria, aunque tengan atributos idénticos.

17. Se ejecutan las instrucciones siguientes. ¿Qué mostrará la consola y por qué?

```
class Contador2 { static total = 0; constructor() { Contador2.total++; } }
new Contador2(); new Contador2(); new Contador2();
console.log(Contador2.total);
```

A. 1, cada instancia guarda su propio total
B. 0, solo se modifica fuera del constructor
C. undefined, accesible solo desde una instancia
D. 3, total pertenece a la clase y las tres instancias lo comparten

Respuesta correcta: D. Los miembros estáticos (`static`) están ligados a la clase y no a las instancias de forma individual. Cada llamada a `new Contador2()` ejecuta el constructor que incrementa en 1 la propiedad estática compartida, acumulando 3.

18. En el draw() del sistema de partículas, el arreglo se recorre con for (let i = particulas.length - 1; i >= 0; i--) y se usa splice(i, 1) para eliminar las partículas muertas. Si se recorriera de adelante hacia atrás, ¿qué problema aparecería?

A. Se omitirían partículas, al eliminar una los elementos se desplazan
B. Se eliminarían partículas vivas ignorando la condición
C. El arreglo se vaciaría por completo
D. splice() deja de funcionar con índices decrecientes

Respuesta correcta: A. Al recorrer de 0 a N y eliminar un elemento con `splice(i, 1)`, los elementos posteriores sufren un reordenamiento de índices hacia la izquierda (-1). Como el contador `i` avanza a la derecha, el elemento que ocupó la posición eliminada jamás será evaluado.

19. Un estudiante escribe let p = new Particula(10, 10); al nivel superior del archivo, antes de setup(), y el constructor de la clase usa createVector() y random(). El sketch falla. ¿Cuál es la causa más probable?

A. Una clase no puede instanciarse fuera de una función
B. Esas funciones de p5.js no están disponibles hasta que la biblioteca se inicializa
C. createVector() solo se invoca desde métodos estáticos
D. El operador new exige ser llamado dentro de draw()

Respuesta correcta: B. En el nivel global antes de `setup()`, la librería p5.js aún no ha terminado de inicializar sus funciones en el entorno de ejecución. Por ende, llamar a `createVector()` o `random()` arroja un error de función no definida.

20. El método esAnomala() de LecturaSensor devuelve this.temperatura > 35 || this.humedad < 20. Se registra new LecturaSensor("Invernadero-D", 35, 20). ¿Cómo se clasifica esa lectura al llamar describir()?

A. ANÓMALA, porque la temperatura alcanza los 35 °C
B. ANÓMALA, porque la humedad llega al 20 %
C. normal, comparaciones estrictas y ninguna se cumple
D. normal, con || deben cumplirse ambas condiciones

Respuesta correcta: C. Los operadores de comparación son estrictos (> y <). Evaluando: `35 > 35` es `false` y `20 < 20` es `false`. `false || false` retorna `false`, clasificando la lectura como normal.

SECCIÓN II. RESPUESTA ABIERTA (PREGUNTAS 21 A 23)

21. Comparación: Enfoque Procedural (Arreglos Paralelos) vs. Orientado a Objetos (POO)

Compare el enfoque procedural con arreglos paralelos y el enfoque orientado a objetos con una clase Particula, considerando la organización del código, la incorporación de nuevas características y la eliminación de elementos. Argumente cuál conviene para un sketch con cien figuras.

- **Organización del código:** En arreglos paralelos hay N arreglos globales inconexos (`x[]`, `y[]`, `vx[]`, `color[]`) vinculados únicamente por la coincidencia del índice. En POO, la clase encapsula todos los datos y comportamientos en una única entidad, almacenando solo un arreglo de objetos (`particulas[]`).
- **Nuevas características:** En arreglos paralelos, agregar una propiedad exige crear un nuevo arreglo global, modificar la rutina de creación e inicialización, y alterar todas las funciones que lo procesan. En POO, basta con agregar una línea en el `constructor()` de la clase.
- **Eliminación de elementos:** Para eliminar un elemento en arreglos paralelos se debe hacer `splice()` coordinadamente sobre cada uno de los N arreglos. En POO se aplica un único `splice()` sobre el arreglo principal.
- **Conclusión/Argumentación:** Para cien figuras conviene rotundamente el enfoque de POO. Reduce radicalmente la complejidad, elimina errores de desincronización de índices y mantiene un ciclo `draw()` sumamente limpio: `for (let p of particulas) { p.actualizar(); p.mostrar(); }`.

22. Proceso Paso a Paso de Instanciación e Independencia de Estado

Explique qué ocurre, paso a paso, cuando se ejecuta new Burbuja(120, 300, 24, "turquesa"), relacionando clase, constructor, objeto e instancia. Justifique por qué dos instancias comparten métodos pero tienen estados distintos.

1. El operador new reserva una porción nueva de memoria para un objeto en blanco.
2. Vincula el prototipo del nuevo objeto con las definiciones de la clase `Burbuja`.
3. Se invoca el método `constructor()`, enlazando la palabra clave `this` a dicho objeto.
4. El constructor inicializa las propiedades (`this.x = 120`, `this.y = 300`, etc.).
5. El operador new devuelve la instancia completamente configurada.

Compartición de Métodos vs. Estado: Los métodos se almacenan una sola vez en el prototipo común (`Burbuja.prototype`) para optimizar memoria. En cambio, los atributos se guardan individualmente en el bloque de memoria propio de cada instancia, permitiendo que cada objeto tenga valores únicos.

SECCIÓN II. RESPUESTA ABIERTA (PREGUNTAS 24 Y 25)

23. Pérdida del Contexto `this` en Callbacks y Soluciones

Analice por qué un método pierde la referencia a this cuando se separa del objeto (por ejemplo, al pasarlo como función de respuesta a un evento). Describa las dos soluciones habituales y explique en qué se diferencian.

Causa: En JS, el valor de `this` depende de *cómo* se llama la función en tiempo de ejecución. Al pasar un método como callback a un evento (como un botón de p5.js), se ejecuta desvinculado de su objeto contenedor, por lo que `this` evalúa a `undefined` o al objeto global.

Soluciones:

- `.bind(this)`: Método de función que crea explícitamente un nuevo wrapper fijando `this` al objeto especificado de forma permanente.
- **Funciones Flecha (`() => {}`):** No poseen un `this` propio, sino que capturan el `this` de su contexto léxico (el ámbito donde fueron creadas originalmente).

24. Variables Locales, Parámetros y Atributos de Instancia

Diferencie variable local, parámetro y atributo en cuanto a dónde se declaran, su alcance y su duración. Ilustre la diferencia con la clase Burbuja y argumente por qué confundirlos produce errores.

- **Variable Local:** Declarada dentro de un bloque con `let/const`. Alcance: restringido a ese bloque. Duración: hasta que el bloque termina.
- **Parámetro:** Declarado en la firma de un método/constructor. Alcance: todo el cuerpo del método. Duración: dura la ejecución del método.
- **Atributo:** Declarado en la clase mediante `this.nombre`. Alcance: accesible en toda la instancia usando `this`. Duración: toda la vida útil del objeto.

Error típico: Si en `Burbuja` escribimos `x += velocidad` omitiendo `this.`, el programa buscará variables locales/globales sin modificar las propiedades reales de la instancia, congelando su comportamiento visual.

25. Principio de Responsabilidad Única: `actualizar()` vs `mostrar()`

Explique el principio de responsabilidad única y la separación entre actualizar() y mostrar() dentro de una clase. Analice cómo esta organización mantiene ordenado el ciclo de draw() y qué problemas aparecerían en una clase que lo hace todo.

Separar la física de un objeto (`actualizar()`) de su representación visual (`mostrar()`) garantiza la modularidad. En `draw()`, permite ejecutar primero un bucle de simulación lógica e interactividad y posteriormente realizar el renderizado gráfico. Si una sola clase mezclara físicas, dibujo, carga de archivos y lógica de interfaz, cualquier cambio en la apariencia podría romper la simulación física, impidiendo además reutilizar la clase en entornos sin interfaz o realizar pruebas unitarias del modelo.

SECCIÓN III. APLICACIÓN MEDIANTE EJEMPLOS (PREGUNTAS 26 A 30)

26. Modelado de Clase Empresarial: `Producto`

Proponga un ejemplo real de una empresa en el que convenga modelar con una clase. Identifique la clase, 3 atributos y 3 métodos clasificados.

Clase: `ProductoInventario` | **Atributos:** `codigoSKU`, `cantidadStock`, `precioUnitario`.

- `descontarStock(cantidad)`: **Modifica el estado** (reduce `this.cantidadStock`).
- `calcularValorTotal()`: **Consulta / Calcula** (retorna `this.cantidadStock * this.precioUnitario`).
- `enviarAlertaReposicion()`: **Efecto externo** (emite un e-mail a compras si el stock es bajo).

27. Proceso ETL y Método de Validación Encapsulado

Proponga un método de validación como esAnomala() en ETL para la clase RegistroVenta y explique las ventajas de ubicarlo dentro de la clase.

```
class RegistroVenta {
  constructor(idVenta, monto, clienteId) {
    this.idVenta = idVenta; this.monto = monto; this.clienteId = clienteId;
  }
  requiereAuditoria() {
    return this.monto <= 0 || this.monto > 10000000 || !this.clienteId;
  }
}
```

Ventaja: Encapsula la regla de negocio. Si las políticas de auditoría cambian, solo se actualiza este método centralizado, evitando duplicar bloques `if` por todo el sistema ETL.

28. Visualización de Datos impulsada por Objetos

Describa un caso de visualización de datos en el que un arreglo de objetos alimente un sketch.

Caso: Mapa interactivo de contaminación del aire. **Clase:** `EstacionMonitoreo` con atributos `latitud`, `longitud` y `nivelPM25`. En `setup()` se cargan los datos y se instancian en un arreglo `estaciones[]`. En `draw()` se recorre el arreglo invocando `mostrar()`. El método usa el atributo `nivelPM25` para definir el color con `lerpColor(verde, rojo, nivel)` y el tamaño del radio del círculo indicador.

29. Asignación por Referencia vs. Clonación en Sistemas de Información

Proponga una situación real donde asignar un objeto a otra variable (const b = a;) provoque un error de referencias.

Escenario: En un módulo contable de duplicación de facturas para un borrador: `const borrador = facturaOriginal; borrador.monto = 0;`. Al ser una copia de referencia, alterar el borrador modifica también la factura auditada en el historial. **Solución:** Instanciar un objeto nuevo e independiente: `const borrador = new Factura(facturaOriginal.cliente, facturaOriginal.monto);`.

30. Conteo de Instancias con Miembros Estáticos

Proponga cómo usaría un miembro estático para contar cuántos clientes se crean e identifique otro ejemplo de información compartida.

```
class Cliente {
  static totalRegistrados = 0;
  constructor(nombre) {
    this.nombre = nombre;
    Cliente.totalRegistrados++;
  }
}
```

No debe ser de instancia porque se reiniciaría individualmente en cada objeto. La propiedad estática le pertenece a la clase global. **Otro ejemplo de uso estático:** Constantes globales como `Cliente.IMPUESTO_APLICABLE = 0.19` o métodos utilitarios como `Cliente.validarEmail(correo)`.